Manage huggingface_hub cache-system

Understand caching

The Hugging Face Hub cache-system is designed to be the central cache shared across libraries that depend on the Hub. It has been updated in v0.8.0 to prevent re-downloading same files between revisions.

The caching system is designed as follows:

```
<CACHE_DIR>
├─ <MODELS>
├─ <DATASETS>
├─ <SPACES>
```

The `<CACHE_DIR>` is usually your user's home directory. However, it is customizable with the `cache_dir` argument on all methods, or by specifying either `HF_HOME` or `HF_HUB_CACHE` environment variable.

Models, datasets and spaces share a common root. Each of these repositories contains the repository type, the namespace (organization or username) if it exists and the repository name:

```
<CACHE_DIR>
├─ models--julien-c--EsperBERTo-small
├─ models--lysandrejik--arxiv-nlp
├─ models--bert-base-cased
├─ datasets--glue
├─ datasets--huggingface--DataMeasurementsFiles
├─ spaces--dalle-mini--dalle-mini
```

It is within these folders that all files will now be downloaded from the Hub. Caching ensures that a file isn't downloaded twice if it already exists and wasn't updated; but if it was updated, and you're asking for the latest file, then it will download the latest file (while keeping the previous file intact in case you need it again).

In order to achieve this, all folders contain the same skeleton:

```
<CACHE_DIR>
├─ datasets--glue
│  ├─ refs
│  ├─ blobs
│  ├─ snapshots
...
```

Each folder is designed to contain the following:

Refs

The `refs` folder contains files which indicates the latest revision of the given reference. For example, if we have previously fetched a file from the `main` branch of a repository, the `refs` folder will contain a file named `main`, which will itself contain the commit identifier of the current head.

If the latest commit of `main` has `aaaaaa` as identifier, then it will contain `aaaaaa`.

If that same branch gets updated with a new commit, that has `bbbbbb` as an identifier, then re-downloading a file from that reference will update the `refs/main` file to contain `bbbbbb`.

Blobs

The `blobs` folder contains the actual files that we have downloaded. The name of each file is their hash.

Snapshots

The `snapshots` folder contains symlinks to the blobs mentioned above. It is itself made up of several folders: one per known revision!

In the explanation above, we had initially fetched a file from the `aaaaaa` revision, before fetching a file from the `bbbbbb` revision. In this situation, we would now have two folders in the `snapshots` folder: `aaaaaa` and `bbbbbb`.

In each of these folders, live symlinks that have the names of the files that we have downloaded. For example, if we had downloaded the `README.md` file at revision `aaaaaa`, we would have the following path:

```
<CACHE_DIR>/<REPO_NAME>/snapshots/aaaaaa/README.md
```

That `README.md` file is actually a symlink linking to the blob that has the hash of the file.

By creating the skeleton this way we open the mechanism to file sharing: if the same file was fetched in revision `bbbbbb`, it would have the same hash and the file would not need to be re-downloaded.

.no_exist (advanced)

In addition to the `blobs`, `refs` and `snapshots` folders, you might also find a `.no_exist` folder in your cache. This folder keeps track of files that you've tried to download once but don't exist on the Hub. Its structure is the same as the `snapshots` folder with 1 subfolder per known revision:

```
<CACHE_DIR>/<REPO_NAME>/.no_exist/aaaaaa/config_that_does_not_exist.json
```

Unlike the `snapshots` folder, files are simple empty files (no symlinks). In this example, the file `"config_that_does_not_exist.json"` does not exist on the Hub for the revision `"aaaaaa"`. As it only stores empty files, this folder is neglectable is term of disk usage.

So now you might wonder, why is this information even relevant? In some cases, a framework tries to load optional files for a model. Saving the non-existence of optional files makes it faster to load a model as it saves 1 HTTP call per possible optional file. This is for example the case in `transformers` where each tokenizer can support additional files. The first time you load the tokenizer on your machine, it will cache which optional files exists (and which doesn't) to make the loading time faster for the next initializations.

To test if a file is cached locally (without making any HTTP request), you can use the try_to_load_from_cache() helper. It will either return the filepath (if exists and cached), the object `_CACHED_NO_EXIST` (if non-existence is cached) or `None` (if we don't know).

```
from huggingface_hub import try_to_load_from_cache, _CACHED_NO_EXIST

filepath = try_to_load_from_cache()
if isinstance(filepath, str):
    # file exists and is cached
    ...
elif filepath is _CACHED_NO_EXIST:
    # non-existence of file is cached
    ...
else:
    # file is not cached
    ...
```

In practice

In practice, your cache should look like the following tree:

```
[  96]  .
└── [ 160]  models--julien-c--EsperBERTo-small
    ├── [ 160]  blobs
    │   ├── [321M]  403450e234d65943a7dcf7e05a771ce3c92faa84dd07db4ac20f592037a1e4bd
    │   ├── [ 398]  7cb18dc9bafbfcf74629a4b760af1b160957a83e
    │   └── [1.4K]  d7edf6bd2a681fb0175f7735299831ee1b22b812
    ├── [  96]  refs
    │   └── [  40]  main
    └── [ 128]  snapshots
        ├── [ 128]  2439f60ef33a0d46d85da5001d52aeda5b00ce9f
        │   ├── [  52]  README.md -> ../../blobs/d7edf6bd2a681fb0175f7735299831ee1b22b812
        │   └── [  76]  pytorch_model.bin -> ../../blobs/403450e234d65943a7dcf7e05a771ce3c92faa84dd07db4ac20f59
        └── [ 128]  bbc77c8132af1cc5cf678da3f1ddf2de43606d48
            ├── [  52]  README.md -> ../../blobs/7cb18dc9bafbfcf74629a4b760af1b160957a83e
            └── [  76]  pytorch_model.bin -> ../../blobs/403450e234d65943a7dcf7e05a771ce3c92faa84dd07db4ac20f59
```

Limitations

In order to have an efficient cache-system, `huggingface-hub` uses symlinks. However, symlinks are not supported on all machines. This is a known limitation especially on Windows. When this is the case, `huggingface_hub` do not use the `blobs/` directory but directly stores the files in the `snapshots/` directory instead. This workaround allows users to download and cache files from the Hub exactly the same way. Tools to inspect and delete the cache (see below) are also supported. However, the cache-system is less efficient as a single file might be downloaded several times if multiple revisions of the same repo is downloaded.

If you want to benefit from the symlink-based cache-system on a Windows machine, you either need to activate Developer Mode or to run Python as an administrator.

When symlinks are not supported, a warning message is displayed to the user to alert them they are using a degraded version of the cache-system. This warning can be disabled by setting the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable to true.

Caching assets

In addition to caching files from the Hub, downstream libraries often requires to cache other files related to HF but not handled directly by `huggingface_hub` (example: file downloaded from GitHub, preprocessed data, logs,…). In order to cache those files, called `assets`, one can use cached_assets_path(). This small helper generates paths in the HF cache in a unified way based on the name of the library requesting it and optionally on a namespace and a subfolder name. The goal is to let every downstream libraries manage its assets its own way (e.g. no rule on the structure) as long as it stays in the right assets folder. Those libraries can then leverage tools from `huggingface_hub` to manage the cache, in particular scanning and deleting parts of the assets from a CLI command.

```
from huggingface_hub import cached_assets_path

assets_path = cached_assets_path(library_name="datasets", namespace="SQuAD", subfolder="download")
something_path = assets_path / "something.json" # Do anything you like in your assets folder !
```

> cached_assets_path() is the recommended way to store assets but is not mandatory. If your library already uses its own cache, feel free to use it!

Assets in practice

In practice, your assets cache should look like the following tree:

```
assets/
└── datasets/
|   ├── SQuAD/
|   |   ├── downloaded/
|   |   ├── extracted/
|   |   └── processed/
|   ├── Helsinki-NLP--tatoeba_mt/
|       ├── downloaded/
|       ├── extracted/
|       └── processed/
└── transformers/
    ├── default/
    |   ├── something/
    ├── bert-base-cased/
    |   ├── default/
    |   └── training/
hub/
└── models--julien-c--EsperBERTo-small/
    ├── blobs/
    |   ├── (...)
    |   ├── (...)
    ├── refs/
    |   └── (...)
    └── [ 128]  snapshots/
        ├── 2439f60ef33a0d46d85da5001d52aeda5b00ce9f/
        |   ├── (...)
        └── bbc77c8132af1cc5cf678da3f1ddf2de43606d48/
            └── (...)
```

Scan your cache

At the moment, cached files are never deleted from your local directory: when you download a new revision of a branch, previous files are kept in case you need them again. Therefore it can be useful to scan your cache directory in order to know

which repos and revisions are taking the most disk space. `huggingface_hub` provides an helper to do so that can be used via `huggingface-cli` or in a python script.

Scan cache from the terminal

The easiest way to scan your HF cache-system is to use the `scan-cache` command from `huggingface-cli` tool. This command scans the cache and prints a report with information like repo id, repo type, disk usage, refs and full local path.

The snippet below shows a scan report in a folder in which 4 models and 2 datasets are cached.

```
➜ huggingface-cli scan-cache
REPO ID                     REPO TYPE SIZE ON DISK NB FILES LAST_ACCESSED LAST_MODIFIED REFS                LOCAL P
--------------------------- --------- ------------ -------- ------------- ------------- ------------------- -------
glue                        dataset         116.3K       15 4 days ago    4 days ago    2.4.0, main, 1.17.0 /home/w
google/fleurs               dataset          64.9M        6 1 week ago    1 week ago    refs/pr/1, main     /home/w
Jean-Baptiste/camembert-ner model           441.0M        7 2 weeks ago   16 hours ago  main                /home/w
bert-base-cased             model             1.9G       13 1 week ago    2 years ago                       /home/w
t5-base                     model            10.1K        3 3 months ago  3 months ago  main                /home/w
t5-small                    model           970.7M       11 3 days ago    3 days ago    refs/pr/1, main     /home/w

Done in 0.0s. Scanned 6 repo(s) for a total of 3.4G.
Got 1 warning(s) while scanning. Use -vvv to print details.
```

To get a more detailed report, use the `--verbose` option. For each repo, you get a list of all revisions that have been downloaded. As explained above, the files that don't change between 2 revisions are shared thanks to the symlinks. This means that the size of the repo on disk is expected to be less than the sum of the size of each of its revisions. For example, here `bert-base-cased` has 2 revisions of 1.4G and 1.5G but the total disk usage is only 1.9G.

```
➜ huggingface-cli scan-cache -v
REPO ID                     REPO TYPE REVISION                                 SIZE ON DISK NB FILES LAST_MODIFIED
--------------------------- --------- ---------------------------------------- ------------ -------- -------------
glue                        dataset   9338f7b671827df886678df2bdd7cc7b4f36dffd        97.7K       14 4 days ago
glue                        dataset   f021ae41c879fcabcf823648ec685e3fead91fe7        97.8K       14 1 week ago
google/fleurs               dataset   129b6e96cf1967cd5d2b9b6aec75ce6cce7c89e8        25.4K        3 2 weeks ago
google/fleurs               dataset   24f85a01eb955224ca3946e70050869c56446805        64.9M        4 1 week ago
Jean-Baptiste/camembert-ner model     dbec8489a1c44ecad9da8a9185115bccabd799fe       441.0M        7 16 hours ago
bert-base-cased             model     378aa1bda6387fd00e824948ebe3488630ad8565         1.5G        9 2 years ago
bert-base-cased             model     a8d257ba9925ef39f3036bfc338acf5283c512d9         1.4G        9 3 days ago
t5-base                     model     23aa4f41cb7c08d4b05c8f327b22bfa0eb8c7ad9        10.1K        3 1 week ago
t5-small                    model     98ffebbb27340ec1b1abd7c45da12c253ee1882a       726.2M        6 1 week ago
t5-small                    model     d0a119eedb3718e34c648e594394474cf95e0617       485.8M        6 4 weeks ago
t5-small                    model     d78aea13fa7ecd06c29e3e46195d6341255065d5       970.7M        9 1 week ago

Done in 0.0s. Scanned 6 repo(s) for a total of 3.4G.
Got 1 warning(s) while scanning. Use -vvv to print details.
```

Grep example

Since the output is in tabular format, you can combine it with any `grep`-like tools to filter the entries. Here is an example to filter only revisions from the "t5-small" model on a Unix-based machine.

```
➜ eval "huggingface-cli scan-cache -v" | grep "t5-small"
t5-small                    model     98ffebbb27340ec1b1abd7c45da12c253ee1882a       726.2M        6 1 week ago
```

```
t5-small                    model     d0a119eedb3718e34c648e594394474cf95e0617       485.8M        6 4 weeks ago
t5-small                    model     d78aea13fa7ecd06c29e3e46195d6341255065d5       970.7M        9 1 week ago
```

Scan cache from Python

For a more advanced usage, use scan_cache_dir() which is the python utility called by the CLI tool.

You can use it to get a detailed report structured around 4 dataclasses:

- HFCacheInfo: complete report returned by scan_cache_dir()
- CachedRepoInfo: information about a cached repo
- CachedRevisionInfo: information about a cached revision (e.g. “snapshot”) inside a repo
- CachedFileInfo: information about a cached file in a snapshot

Here is a simple usage example. See reference for details.

```
>>> from huggingface_hub import scan_cache_dir

>>> hf_cache_info = scan_cache_dir()
HFCacheInfo(
    size_on_disk=3398085269,
    repos=frozenset({
        CachedRepoInfo(
            repo_id='t5-small',
            repo_type='model',
            repo_path=PosixPath(...),
            size_on_disk=970726914,
            nb_files=11,
            last_accessed=1662971707.3567169,
            last_modified=1662971107.3567169,
            revisions=frozenset({
                CachedRevisionInfo(
                    commit_hash='d78aea13fa7ecd06c29e3e46195d6341255065d5',
                    size_on_disk=970726339,
                    snapshot_path=PosixPath(...),
                    # No `last_accessed` as blobs are shared among revisions
                    last_modified=1662971107.3567169,
                    files=frozenset({
                        CachedFileInfo(
                            file_name='config.json',
                            size_on_disk=1197
                            file_path=PosixPath(...),
                            blob_path=PosixPath(...),
                            blob_last_accessed=1662971707.3567169,
                            blob_last_modified=1662971107.3567169,
                        ),
                        CachedFileInfo(...),
                        ...
                    }),
                ),
                CachedRevisionInfo(...),
                ...
            }),
        ),
        CachedRepoInfo(...),
        ...
    }),
    warnings=[
```

```
        CorruptedCacheException("Snapshots dir doesn't exist in cached repo: ..."),
        CorruptedCacheException(...),
        ...
    ],
)
```

Clean your cache

Scanning your cache is interesting but what you really want to do next is usually to delete some portions to free up some space on your drive. This is possible using the `delete-cache` CLI command. One can also programmatically use the delete_revisions() helper from HFCacheInfo object returned when scanning the cache.

Delete strategy

To delete some cache, you need to pass a list of revisions to delete. The tool will define a strategy to free up the space based on this list. It returns a DeleteCacheStrategy object that describes which files and folders will be deleted. The DeleteCacheStrategy allows give you how much space is expected to be freed. Once you agree with the deletion, you must execute it to make the deletion effective. In order to avoid discrepancies, you cannot edit a strategy object manually.

The strategy to delete revisions is the following:

- the `snapshot` folder containing the revision symlinks is deleted.
- blobs files that are targeted only by revisions to be deleted are deleted as well.
- if a revision is linked to 1 or more `refs`, references are deleted.
- if all revisions from a repo are deleted, the entire cached repository is deleted.

> Revision hashes are unique across all repositories. This means you don't need to provide any `repo_id` or `repo_type` when removing revisions.

> If a revision is not found in the cache, it will be silently ignored. Besides, if a file or folder cannot be found while trying to delete it, a warning will be logged but no error is thrown. The deletion continues for other paths contained in the DeleteCacheStrategy object.

Clean cache from the terminal

The easiest way to delete some revisions from your HF cache-system is to use the `delete-cache` command from `huggingface-cli` tool. The command has two modes. By default, a TUI (Terminal User Interface) is displayed to the user to select which revisions to delete. This TUI is currently in beta as it has not been tested on all platforms. If the TUI doesn't work on your machine, you can disable it using the `--disable-tui` flag.

Using the TUI

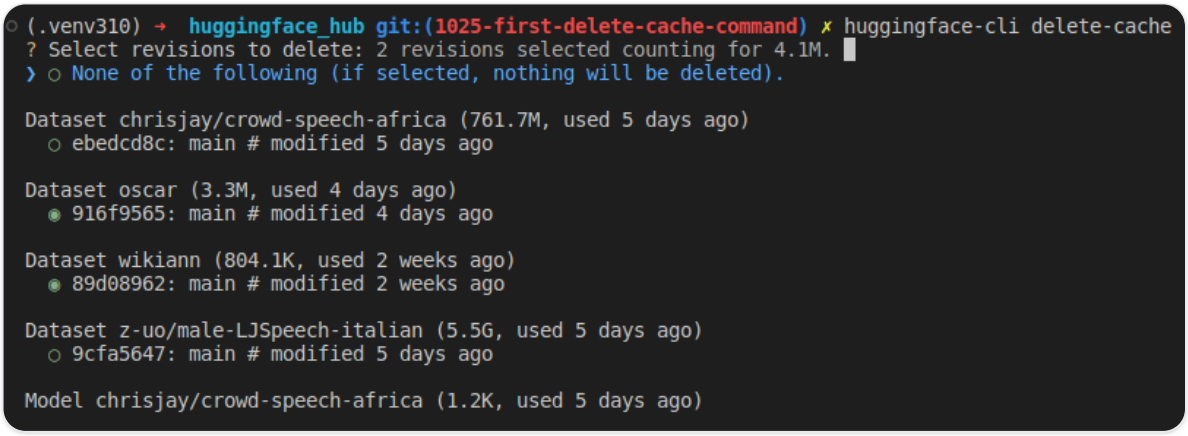
This is the default mode. To use it, you first need to install extra dependencies by running the following command:

```
pip install huggingface_hub["cli"]
```

Then run the command:

```
huggingface-cli delete-cache
```

You should now see a list of revisions that you can select/deselect:



Instructions:

- Press keyboard arrow keys `<up>` and `<down>` to move the cursor.
- Press `<space>` to toggle (select/unselect) an item.
- When a revision is selected, the first line is updated to show you how much space will be freed.
- Press `<enter>` to confirm your selection.
- If you want to cancel the operation and quit, you can select the first item (“None of the following”). If this item is selected, the delete process will be cancelled, no matter what other items are selected. Otherwise you can also press `<ctrl+c>` to quit the TUI.

Once you’ve selected the revisions you want to delete and pressed `<enter>`, a last confirmation message will be prompted. Press `<enter>` again and the deletion will be effective. If you want to cancel, enter `n`.

```
✗ huggingface-cli delete-cache --dir ~/.cache/huggingface/hub
? Select revisions to delete: 2 revision(s) selected.
? 2 revisions selected counting for 3.1G. Confirm deletion ? Yes
Start deletion.
Done. Deleted 1 repo(s) and 0 revision(s) for a total of 3.1G.
```

Without TUI

As mentioned above, the TUI mode is currently in beta and is optional. It may be the case that it doesn’t work on your machine or that you don’t find it convenient.

Another approach is to use the `--disable-tui` flag. The process is very similar as you will be asked to manually review the list of revisions to delete. However, this manual step will not take place in the terminal directly but in a temporary file generated on the fly and that you can manually edit.

This file has all the instructions you need in the header. Open it in your favorite text editor. To select/deselect a revision, simply comment/uncomment it with a `#`. Once the manual review is done and the file is edited, you can save it. Go back to your terminal and press `<enter>`. By default it will compute how much space would be freed with the updated list of revisions. You can continue to edit the file or confirm with `"y"`.

```
huggingface-cli delete-cache --disable-tui
```

Example of command file:

```
# INSTRUCTIONS
# ------------
# This is a temporary file created by running `huggingface-cli delete-cache` with the
# `--disable-tui` option. It contains a set of revisions that can be deleted from your
# local cache directory.
#
# Please manually review the revisions you want to delete:
#   - Revision hashes can be commented out with '#'.
#   - Only non-commented revisions in this file will be deleted.
#   - Revision hashes that are removed from this file are ignored as well.
#   - If `CANCEL_DELETION` line is uncommented, the all cache deletion is cancelled and
#     no changes will be applied.
#
# Once you've manually reviewed this file, please confirm deletion in the terminal. This
# file will be automatically removed once done.
# ------------

# KILL SWITCH
# ------------
# Un-comment following line to completely cancel the deletion process
# CANCEL_DELETION
# ------------

# REVISIONS
# ------------
# Dataset chrisjay/crowd-speech-africa (761.7M, used 5 days ago)
    ebedcd8c55c90d39fd27126d29d8484566cd27ca # Refs: main # modified 5 days ago

# Dataset oscar (3.3M, used 4 days ago)
#    916f956518279c5e60c63902ebdf3ddf9fa9d629 # Refs: main # modified 4 days ago

# Dataset wikiann (804.1K, used 2 weeks ago)
    89d089624b6323d69dcd9e5eb2def0551887a73a # Refs: main # modified 2 weeks ago

# Dataset z-uo/male-LJSpeech-italian (5.5G, used 5 days ago)
#    9cfa5647b32c0a30d0adfca06bf198d82192a0d1 # Refs: main # modified 5 days ago
```

Clean cache from Python

For more flexibility, you can also use the delete_revisions() method programmatically. Here is a simple example. See reference for details.

```
>>> from huggingface_hub import scan_cache_dir

>>> delete_strategy = scan_cache_dir().delete_revisions(
...     "81fd1d6e7847c99f5862c9fb81387956d99ec7aa"
...     "e2983b237dccf3ab4937c97fa717319a9ca1a96d",
...     "6c0e6080953db56375760c0471a8c5f2929baf11",
... )
>>> print("Will free " + delete_strategy.expected_freed_size_str)
Will free 8.6G

>>> delete_strategy.execute()
Cache deletion done. Saved 8.6G.
```

<> Update on GitHub

← Collections

Model Cards →